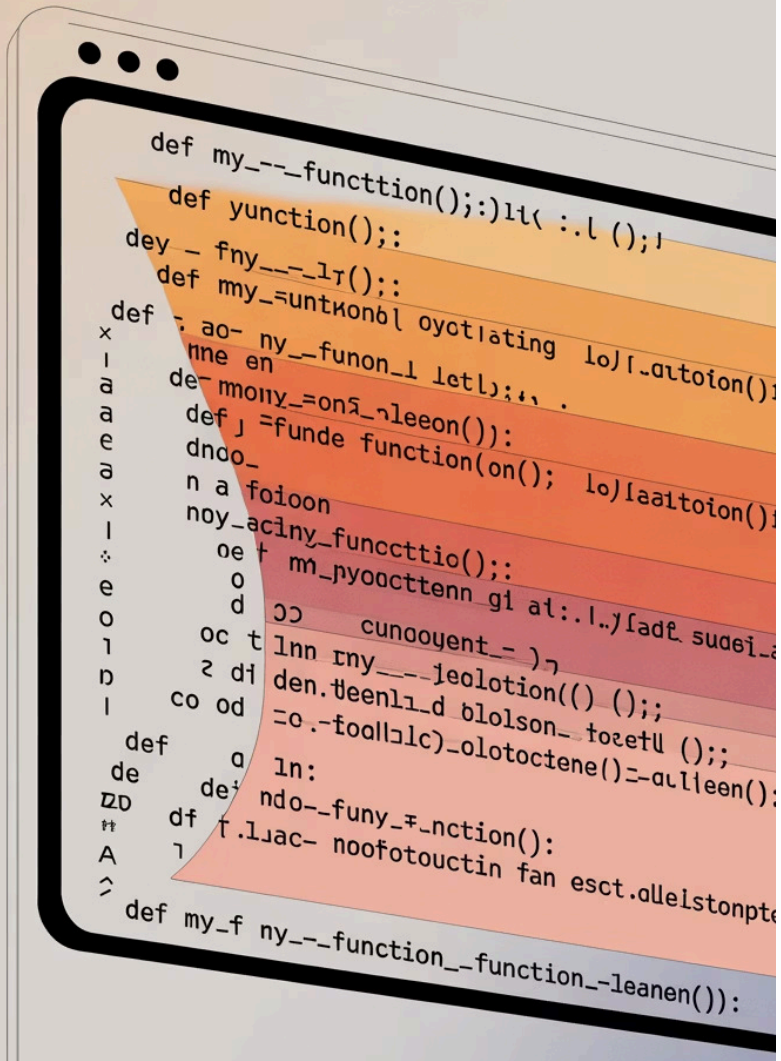


Lección 1: Variables y Tipos de Datos Fundamentales en Python

Bienvenidos al primer módulo de nuestro curso de Python para Análisis de Datos.

En esta sesión exploraremos los conceptos fundamentales que constituyen la base de cualquier análisis de datos: las variables y los tipos de datos.

Made with GAMMA

¿Qué es una Variable?

El Contenedor de Nuestros Datos

Una variable es un **nombre que apunta a un valor almacenado en la memoria** del ordenador. Es como una etiqueta que nos permite identificar y acceder a la información que necesitamos para nuestros análisis.

Sintaxis Básica

Utilizamos el signo de asignación (=) para crear variables:

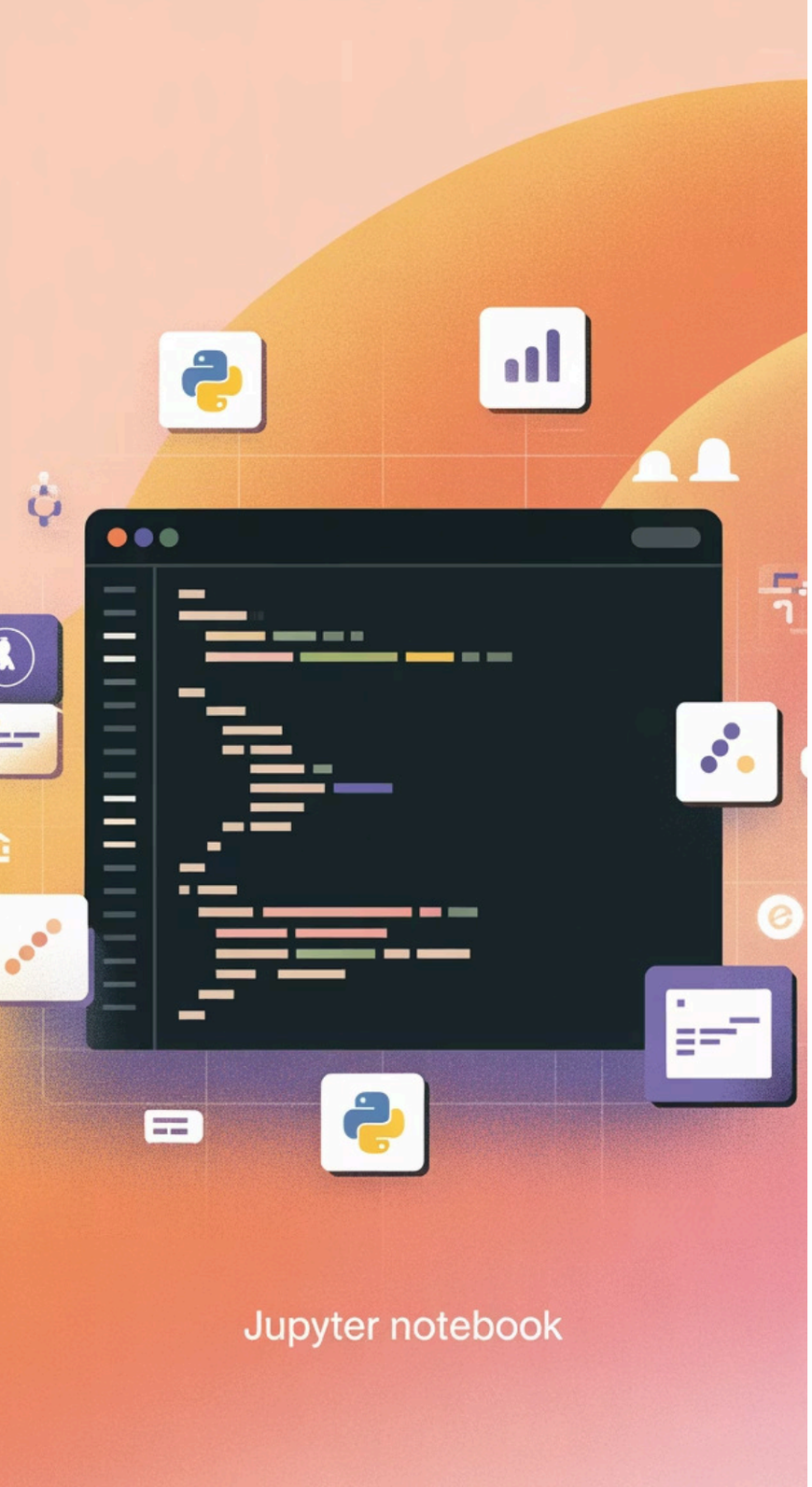
```
salario_base = 45000
ciudad = "Barcelona"
```

Convención de Nomenclatura

- Usar **snake_case** (minúsculas con _ para separar)
- Nombres descriptivos y claros

La nomenclatura clara es fundamental para código mantenible y comprensible.

Made with GAMMA



Práctica en Jupyter Notebook

Pon en práctica tus conocimientos de variables y tipos de datos en un entorno interactivo.

Jupyter Notebook es un entorno de computación interactivo basado en la web que te permite combinar código, texto explicativo, ecuaciones y visualizaciones en un solo documento. Es ideal para aprender Python porque facilita la experimentación y la visualización de resultados paso a paso, lo que es perfecto para afianzar conceptos como las variables y los tipos de datos.

Cómo empezar a practicar

1 Crear una nueva celda

Haz clic en el botón **+** en la barra de herramientas o selecciona "Insertar > Insertar celda debajo" para añadir una nueva área de trabajo.

2 Celdas de Código vs. Markdown

Alterna entre celdas de `Código` (para escribir y ejecutar Python) y `Markdown` (para añadir texto explicativo y formato) usando el menú desplegable en la barra de herramientas.

3 Ejecutar código

Escribe tu código Python en una celda de código y presiona **Shift + Enter** para ejecutarlo. El resultado aparecerá justo debajo.

Made with GAMMA

Ejemplos prácticos de código

Copia y pega estos ejemplos en tus celdas de código para verlos en acción:

1. Crear variables de diferentes tipos

```
nombre = "Alice"
edad = 30
altura = 1.75
es_estudiante = True
```

2. Usar `print()` para ver resultados

```
print(nombre)
print(f"Hola, mi nombre es {nombre} y tengo {edad} años.")
```

3. Probar operaciones matemáticas

```
resultado_suma = edad + 5
resultado_multi = altura * 2
print(resultado_suma)
print(resultado_multi)
```

4. Experimentar con métodos de strings

```
saludo = "hola mundo"
print(saludo.upper())
print(saludo.replace("mundo", "Python"))
print(saludo.capitalize())
```

Consejos para trabajar en Jupyter

→ Experimenta libremente

No tengas miedo de probar diferentes valores y funciones. Jupyter está diseñado para la experimentación y el aprendizaje interactivo.

→ Documenta tu código

Usa celdas de Markdown para explicar lo que haces en cada paso. Esto te ayudará a entender y recordar tu propio trabajo más adelante.

→ Verifica los resultados

Utiliza `print()` constantemente para ver el estado de tus variables y el resultado de tus operaciones. Es una herramienta clave para depurar.

Ventajas de Jupyter para el aprendizaje

Interactividad Inmediata

Ejecuta código en pequeñas secciones y ve los resultados al instante, lo que facilita la comprensión.

Documentación Integrada

Combina código, texto explicativo y visualizaciones en un solo lugar, creando un "cuaderno" de aprendizaje completo.

Entorno Flexible

Ideal para explorar datos, prototipar algoritmos y presentar tus análisis de forma clara y estructurada.

Made with GAMMA

Tipos de Datos I: Numéricos

Para Contar y Medir

int (Integer)

Números enteros que se utilizan para contar elementos discretos como el número de clientes, cantidad de productos, o años de experiencia.

```
clientes = 5000
productos_vendidos = 1250
```

Perfectos para datos que no requieren decimales.

float (Flotante)

Números decimales esenciales para mediciones precisas, promedios, tasas y métricas que requieren precisión decimal.

```
temperatura = 36.5
tasa_interes = 0.057
precio_producto = 29.99
```

Tip DA: En Python, la división (/) siempre devuelve un float.

Made with GAMMA

Tipos de Datos II: Cadenas de Texto

Para Etiquetas, Nombres y Texto Libre

str (String)

Una **secuencia de caracteres** (letras, números, símbolos) que debe ir siempre encerrada entre comillas simples (' ') o dobles (" ").

```
region = 'Norte'
mensaje = "¡Bienvenido al sistema!"
codigo_postal = "28001"
```

Los strings son fundamentales para almacenar categorías, etiquetas y cualquier información textual en nuestros análisis.

Visualización con print()

La función print() nos permite mostrar el contenido de variables y mensajes en la consola o notebook.

```
print(region)  # Muestra: Norte
print(mensaje) # Muestra: ¡Bienvenido!
```

Made with GAMMA

Combinando Variables y Texto

La Función print() y la Concatenación

1

Método 1: Suma (+)

Une strings **sin espacio automático**. Debemos añadir espacios manualmente para legibilidad.

```
nombre = "Ana"
print("Hola " + nombre)
# Resultado: Hola Ana
```

2

Método 2: Coma (,)

Pasa múltiples argumentos a print(). **Añade espacios automáticamente** entre elementos.

```
precio = 25
print("El producto cuesta", precio, "euros.")
# Resultado: El producto cuesta 25 euros.
```

La concatenación es especialmente útil para crear mensajes dinámicos y reportes automatizados en nuestros análisis de datos.

Made with GAMMA

Operaciones y Resultados

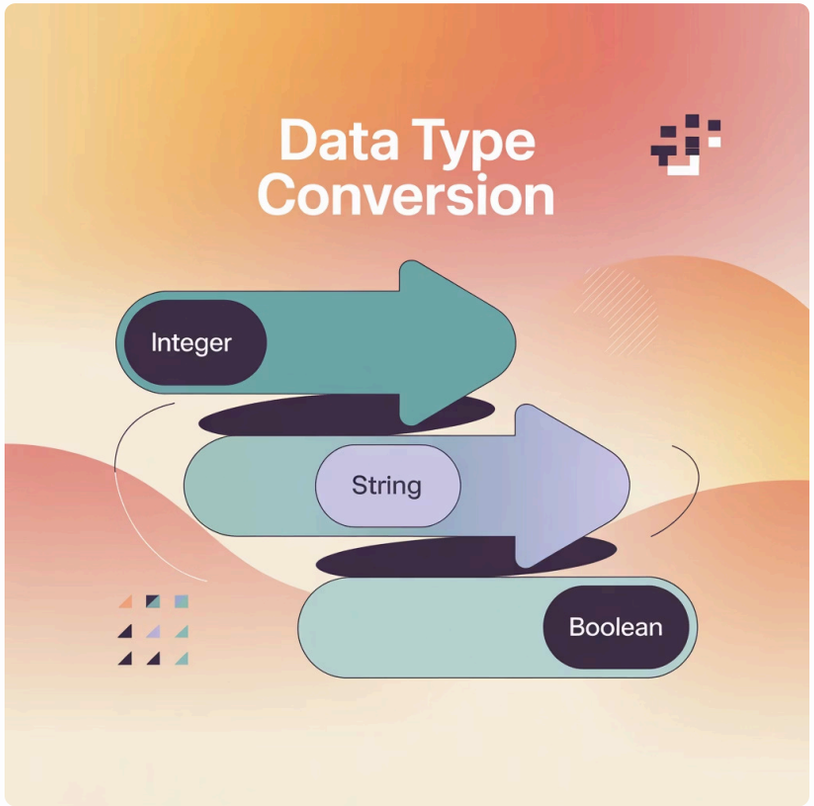
Cálculos Fundamentales en Python

Operador	Descripción	Ejemplo	Resultado
+	Suma	10 + 5	15
-	Resta	10 - 5	5
*	Multiplicación	10 * 5	50
/	División	10 / 4	2.5 (float)
**	Potencia	2 ** 3	8

Made with GAMMA

Conversión de Tipos (Type Casting)

Preparando Datos para el Cálculo



Definición: Forzar a una variable a cambiar de un tipo de dato a otro. Es una herramienta esencial en la limpieza de datos.

Funciones de Conversión

- int(variable): Convierte a entero
- float(variable): Convierte a flotante
- str(variable): Convierte a cadena de texto

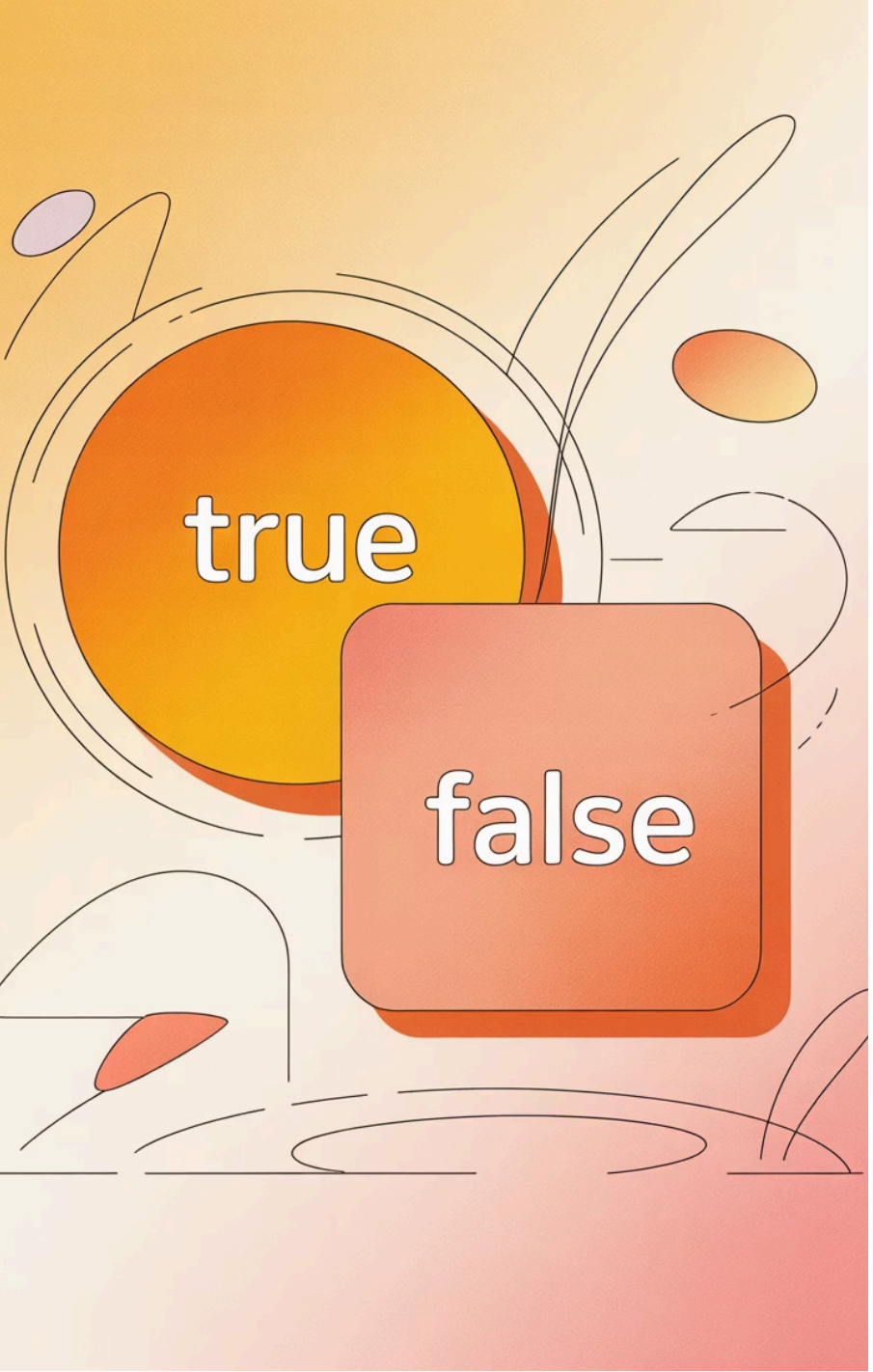
Ejemplo Práctico

Cuando importas un CSV, los números a veces se leen como texto:

```
dato_importado = "1200"  # Esto es un str
monto = int(dato_importado)  # Ahora es 1200 (int)
```

El Type Casting es fundamental cuando trabajamos con datos procedentes de fuentes externas como CSV, APIs o bases de datos, donde los tipos pueden no ser los esperados.

Made with GAMMA



Tipos Lógicos y Validación

El Tipo de Datos bool y Control de Calidad

bool (Booleano)

Representa el **valor lógico**, solo puede ser True (Verdadero) o False (Falso). Se utiliza para condiciones, filtros y control de flujo en nuestros análisis.

```
datos_completos = True
tiene_errores = False
```

Verificación de Tipos

type(variable) nos dice de qué clase es un objeto.

isinstance(variable, tipo) pregunta si una variable es de un tipo específico, devolviendo True o False.

```
type(25)  #
isinstance(25, int)  # True
```

Los booleanos son esenciales para el control de calidad de datos y la implementación de lógica condicional en nuestros scripts de análisis.

Made with GAMMA

Métodos de Strings: Normalización

Limpiando Datos Categóricos y Texto

Los strings vienen con **métodos integrados** que son herramientas de limpieza extremadamente poderosas para el análisis de datos.

.upper()

Convierte TODO a MAYÚSCULAS. Ejemplo: "españa" → "ESPAÑA". Útil para estandarizar códigos de país o categorías importantes.

.lower()

Convierte todo a minúsculas. Ejemplo: "MADRID" → "madrid". Ideal para normalizar direcciones de email o URLs.

.capitalize()

Primera letra en mayúscula. Ejemplo: "ana garcía" → "Ana garcía". Perfecto para nombres propios.

.title()

Cada palabra con mayúscula inicial. Ejemplo: "analista de datos" → "Analista De Datos". Útil para títulos profesionales.

Uso en Data Science: Fundamental para asegurarnos de que la misma categoría de datos (ej. un país) no se cuente varias veces por errores de mayúsculas/minúsculas.

Made with GAMMA

Métodos de Strings: Limpieza y Estructura

Extrayendo Información Útil

Limpieza de Espacios

.strip() elimina espacios en blanco al inicio y al final de una cadena. Esencial para datos importados de fuentes externas.

Reemplazo y Sustitución

.replace('viejo', 'nuevo') sustituye una subcadena por otra. Útil para corregir errores sistemáticos en los datos.

```
texto_sucio = "  Madrid  "
texto_limpio = texto_sucio.strip()
# Resultado: "Madrid"

dato_error = "Barcelon@"
dato_corregido = dato_error.replace("@", "a")
# Resultado: "Barcelona"
```

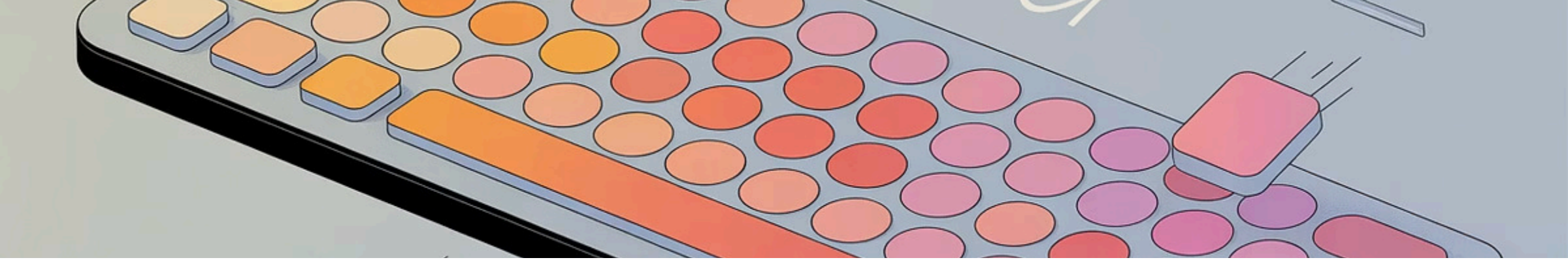
Estructuración de Datos

.split() divide el string en una lista usando el carácter que especifiques.

```
nombre_completo = "Ana,García,28"
datos = nombre_completo.split(",")
# Resultado: ["Ana", "García", "28"]
```

Tip DA: Usa .split() para descomponer datos que vienen juntos en un mismo campo.

Made with GAMMA



input()

¿Cómo Recibimos Datos del Usuario?

1 Función input()

Pausa la ejecución y espera a que el usuario escriba algo y presione Enter. Ideal para scripts interactivos.

```
nombre_user = input("Introduce tu nombre: ")
print("Gracias,", nombre_user)
```

2 ¡ADVERTENCIA CLAVE!

La función input() **siempre devuelve un tipo str** (texto). Si esperas un número, debes usar Type Casting.

```
edad_str = input("¿Cuántos años tienes? ")
edad_int = int(edad_str)  # Conversión necesaria
```

Made with GAMMA

Resumen de Fundamentos de Python (Lección 01)

Hemos explorado los bloques de construcción básicos de Python, esenciales para cualquier tarea de análisis de datos. Aquí te presentamos un resumen infográfico:

Tipos de Datos

Variables `int`, `float`, `str`, `bool` como base de toda información.

Operaciones

Realiza cálculos (+, -, *, /, **) y compara valores para obtener resultados lógicos.

`print()`

Visualiza y combina texto con variables para comunicar tus hallazgos.

Type Casting

Convierte datos entre tipos (`int()`, `str()`, `float()`) para su correcto procesamiento.

Métodos de Strings

Limpia, normaliza y estructura texto para análisis de datos categóricos.

`input()`

Interactúa con el usuario, solicitando datos que siempre serán `str` por defecto.

Made with GAMMA